

Candidate Technical Assessment Package

RKG Portal – Technical Assessment & Architecture Challenge

Role: Full Stack Developer

Scope: End-to-End Build, Concurrency Control, and Architecture Review

Target City Context: Atlanta, GA

Timeframe: 24 Hours from receipt (Target effort: ~4 Hours)

Submission Format: Private GitHub Repository Pull Request + Live Deployed URL

1. Executive Summary & Project Purpose

RKG operates an in-home mobile wellness service across major US cities. We are rebuilding our core booking and operations infrastructure (the **RKG Portal**) to unify customer booking, automated dispatching, payment flows, and provider schedules into a single, reliable system of record.

Our primary technical challenge is delivering a **simple, pragmatic, and highly resilient engine**. We actively avoid overengineered architectures in favor of clean database-level consistency, straightforward API designs, and rapid delivery.

In this assessment, you will build a functional, production-ready slice of the RKG Portal for the **Atlanta, GA** market. You will demonstrate how you translate business requirements into production code, handle real-world backend race conditions, simulate secure payment flows, and document how you leverage AI to accelerate execution.

2. Required Deliverables

Your submission must be delivered via a single GitHub repository containing:

1. **Working Application (Next.js + PostgreSQL)**

- **Customer Booking View:** Public interface allowing a customer in Atlanta, GA to select a service, calculate a 30% deposit, and initiate a booking.
- **Provider / Dispatch View:** Admin/Provider portal showing incoming bookings and enabling acceptance.
- **Simulated Race Condition Button:** A UI or API action that triggers two concurrent provider accepts simultaneously to visually prove your concurrency locking.

2. **Database Schema & Seed Script (/prisma, /migrations, or /scripts)**

- PostgreSQL tables representing Bookings, Providers, Availability, Payments, and Webhook Deduplication.
- Seed file populating at least **2 active Atlanta providers** with overlapping availability schedules.

3. **Automated Testing Suite**

- At least **1 automated test** proving that concurrent accepts for the same time window/provider fail safely without double-booking.
- At least **1 automated test** verifying that redelivered Stripe webhooks are processed

idempotently.

4. Live Public Deployment

- Deployed, publicly accessible web application (e.g., Vercel + Supabase/Neon PostgreSQL).

5. Documentation & Written Artifacts (/docs)

- **CRITIQUE.md**: A 1–2 page evaluation critiquing overengineering in complex specifications and proposing a simplified alternative schema.
- **ARCHITECTURE.md**: System architecture and data-flow diagram explaining your locking and payment idempotency strategy.
- **AI_PROMPTS.md**: Detailed log documenting the prompts, tools (e.g., Cursor, Claude), and workflows you used to build this assignment.
- **README.md**: Clear setup and local run instructions.

3. Functional & Technical Requirements

A. Core Business Logic & State Rules

- **Service Market**: Atlanta, GA.
- **Payment Deposit Flow**:
 - Total price is calculated dynamically based on service duration.
 - System captures a **30% deposit** to move a booking from DRAFT to CONFIRMED_SEARCHING.
 - The remaining **70% balance** is billed upon service start.
- **Atomic Dispatch & Concurrency Control**:
 - Multiple eligible providers may receive a booking offer simultaneously.
 - The system must enforce an **atomic first-accept lock**. Exactly one provider can win a booking.
 - **Database-Level Backstop**: You must enforce schedule exclusivity directly in PostgreSQL (e.g., using database transactions, explicit row locking SELECT ... FOR UPDATE, or a Postgres range-exclusion constraint) so that double-booking is physically impossible even under heavy API load.

B. Payment & Webhook Resilience

- **Mock Stripe Webhook Endpoint**: Build an API handler receiving simulated Stripe payment events (e.g., payment_intent.succeeded).
- **Idempotency Guarantee**: Create a webhook_events deduplication table in PostgreSQL. Redelivered webhooks with the same event ID must return a 200 OK status without re-processing charges or updating state twice.

4. Reference Context: RKG SDS Specification Overview

Below is the abstract of the proposed enterprise SDS specification currently under review. Use this context to inform your implementation and write your **CRITIQUE.md**.

[REFERENCE ARCHITECTURE SUMMARY]

System Architecture: Next.js + Vercel + Supabase (Postgres/PostGIS) + Redis + Stripe + Twilio.

Proposed Data Schema: 24 Tables

- Commerce: Customers, Memberships, Bookings, Services, Payments, GiftCards, GiftCardRedemptions, WebhookEvents, Cities, CorporateAccounts, PrepaidBlocks
- Dispatch & Operations: Providers, ProviderAvailability, ProviderTimeOff, DispatchOffers, ProviderEarnings, Payouts
- Platform & Messaging: StaffUsers, AuditLogs, Flags, MaskedSessions, Messages, Contacts, Notifications

Dispatch State Machine: 16 Booking States

{draft, validating, rejected, payment_pending, payment_failed, confirmed_searching, provider_notified, no_provider_available, provider_assigned, provider_en_route, provider_arrived, provider_no_show, service_started, service_completed, customer_no_show, customer_cancelled}

5. Submission Instructions

1. **Repository:** Push your code to a private GitHub repository and add the RKG hiring team as collaborators.
2. **Pull Request:** Open a Pull Request from a feature branch (e.g., feature/rkg-portal-build) into main to simulate production code delivery.
3. **Live URL:** Include your live deployment link at the top of your repository's README.md.
4. **Interview Walkthrough:** In the subsequent interview round, you will conduct a **20-minute live walkthrough:**
 - Demonstrate the Customer booking flow on your live site.
 - Trigger the concurrent accept race-condition test.
 - Show a failed/redelivered webhook being safely deduplicated.
 - Review your CRITIQUE.md and AI workflow.